

Echoasis

Auteurs :

Tevairua JOUEN, Marius POINTEAU

Janvier 2026

Table des matières

1	Vue d'ensemble des objectifs initiaux	1
1.1	Concept	1
1.2	Piliers techniques initiaux	1
2	Intelligence artificielle	2
2.1	Système Unreal	2
2.1.1	FSM	2
2.1.2	Behavior Tree	2
2.1.3	State Tree	2
2.2	États	2
2.2.1	Choix des états	2
2.2.2	Errer	3
2.2.3	Manger	3
2.2.4	Fuir	4
2.3	Axes d'amélioration	6
2.3.1	Gameplay Ability System	6
3	Simulation d'environnement en temps réel	7
3.1	Description	7
3.2	Modification de terrain	7
3.3	Méthodologie	8
3.3.1	Fonctionnement	8
3.3.2	Représentation et génération du terrain	8
3.3.3	Déformation dynamique du terrain	8
3.3.4	Interaction avec les oasis et les splines	9
3.3.5	Contraintes de performance	10
3.3.6	Analyse critique et limites identifiées	10
3.3.7	Décision	10
3.3.8	Runtime Virtual Texture	10

1 Vue d'ensemble des objectifs initiaux

1.1 Concept

À l'origine, le projet avait pour ambition de proposer un jeu de simulation en temps réel centré sur la création et l'évolution d'un écosystème d'oasis. Le joueur pouvait interagir dynamiquement avec l'environnement, en observant ses transformations, mais aussi en les déclenchant directement.

Cependant, le projet a été confronté à des contraintes liées au temps. En raison de ces limites, l'ensemble des idées initialement envisagées n'a pas pu être entièrement mis en œuvre.

1.2 Piliers techniques initiaux

La vision reposait sur trois grands axes techniques, pensés pour fonctionner ensemble :

1. Simulation d'environnement en temps réel

- Modification dynamique des oasis par des splines modifiables
- Placement de formes prédéfinies
- Déformation du terrain

2. (PCG) Génération des éléments naturels de manière procédurale

- Placement cohérent des points d'intérêt
- Adaptation du contenu généré aux formes et aux transformations du terrain

3. Vie animale et comportements

- Ajout d'une faune dynamique
- Comportements individuels et de groupe
- Interaction avec l'environnement

2 Intelligence artificielle

Pour la faune dynamique, nous avons choisi d'utiliser les systèmes d'agents d'Unreal plutôt que de les créer nous-mêmes, au vu de la scope du projet.

2.1 Système Unreal

Tout d'abord, il a fallu choisir un système d'Unreal pour gérer les comportements des agents. Nous avons déjà vu les Finite State Machines (FSM) et les Behavior Trees, mais nous avons découvert une autre solution : les State Trees.

2.1.1 FSM

Pour commencer, les FSM sont une solution peu efficace puisque le système tourne en boucle dans le tick et est donc peu optimisé pour la gestion d'un grand nombre d'agents.

2.1.2 Behavior Tree

Ensuite, les Behavior Trees sont une version améliorée des FSM, permettant une meilleure gestion des transitions. Cependant, ils sont eux aussi exécutés en boucle dans le tick, ce qui pose toujours des problèmes d'optimisation.

2.1.3 State Tree

Enfin, les State Trees. Nous avons découvert ce système grâce à une vidéo de l'Unreal Fest 2024 qui montrait l'utilisation d'un State Tree pour simuler un écosystème composé de dinosaures, ce qui correspondait bien à notre objectif. Les State Trees reprennent le meilleur des deux solutions précédentes en conservant un système d'états avec des transitions directes, tout en résolvant le problème du tick grâce à un fonctionnement proche des événements. Cette solution offre également une meilleure lisibilité et facilite le debug.

2.2 États

Nous allons maintenant parler des états. Les animaux possèdent différents états, similaires à ceux qu'un vrai animal peut avoir dans la nature, afin de pousser au maximum le réalisme de la simulation.

2.2.1 Choix des états

Étant donné le temps limité, il a fallu choisir un nombre restreint d'états parmi une liste que nous avons définie : errer, manger, fuir, boire, dormir, se reproduire... Nous avons finalement retenu les trois états les plus importants : errer, manger et fuir.

2.2.2 Erreur

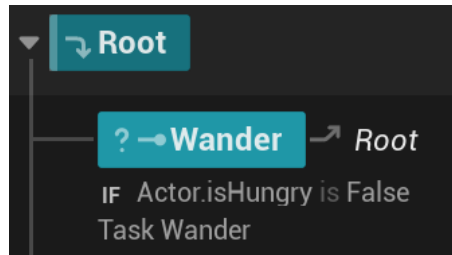


FIGURE 1 – Etat "error"

L'état erreur est le premier que nous avons implémenté. Au début, nous choisissons simplement un point aléatoire dans une zone autour de l'agent. Cependant, après plusieurs essais, nous avons remarqué que l'agent pouvait sélectionner un point en dehors de la Navigation Mesh, ce qui causait des problèmes de déplacement. Pour corriger cela, nous avons forcé le point choisi à être projeté sur le NavMesh en prenant le point valide le plus proche.

De plus, après réflexion sur l'interaction entre les différents états, il nous fallait un moyen d'interrompre le déplacement de l'agent si un état plus prioritaire, comme fuir ou manger, devait être activé. Nous avons donc ajouté un point d'arrêt dans la boucle de déplacement.

2.2.3 Manger

L'état manger est plus complexe que les autres, car il se décompose en plusieurs sous-états et varie selon le type d'animal.

Pour la détection de l'environnement, nous avons mis en place deux systèmes :

le Pawn Sensing pour détecter les proies ou les prédateurs ;

une sphère de détection pour la nourriture.

La sphère de détection enregistre uniquement les éléments correspondant au régime alimentaire de l'animal (herbivore ou carnivore) et conserve la nourriture la plus proche détectée.

En termes de sous-états :

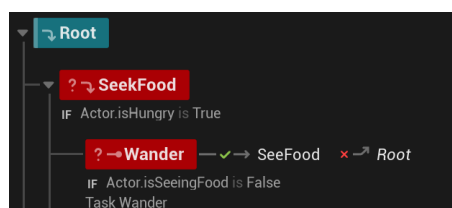


FIGURE 2 – Etat "erreur en ayant faim"

Nous avons d'abord créé une version spécifique de l'état erreur lorsque l'animal a faim. Cette version s'arrête dès que de la nourriture est détectée et transitionne vers l'état de déplacement vers la nourriture ou la proie.

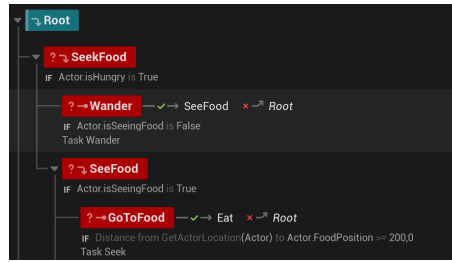


FIGURE 3 – Etat de déplacement vers la nourriture

Vient ensuite l'état de déplacement vers la nourriture, nommé GoToFood. Cet état possède une tâche divisée en deux branches :

Herbivore : l'agent se déplace vers la position du fruit détecté ;

Carnivore : l'agent cible un autre pawn et lance un timer afin que le prédateur puisse abandonner la poursuite s'il n'atteint pas sa cible.

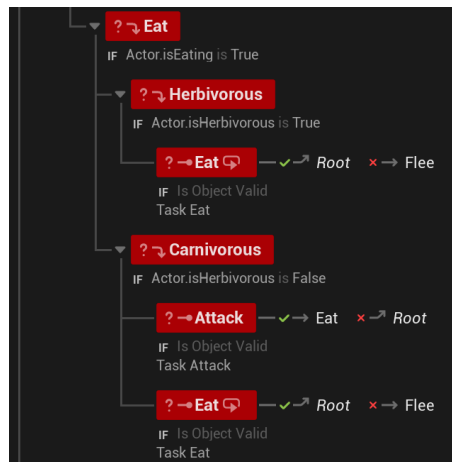


FIGURE 4 – Etat "manger"

L'état manger se divise ensuite en deux sous-états :

Herbivore, qui consomme directement le fruit à portée ;

Carnivore, qui attaque d'abord sa cible avant de la manger.

2.2.4 Fuir

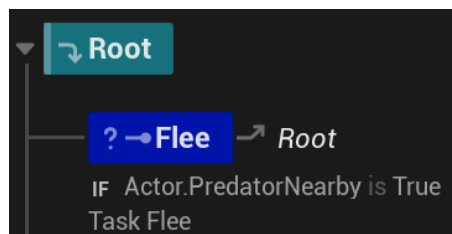


FIGURE 5 – Etat "fuir"

Enfin, l'état fuir. Cet état est prioritaire, quelle que soit la tâche en cours. Cette priorité nous a obligés à repenser l'ensemble des tâches précédemment mises en place. Nous avons donc ajouté une porte de sortie sur toutes les tâches, permettant de revenir à la racine du State Tree si la variable PredatorNearby est vraie.

Le fait qu'Unreal utilise des fonctions latentes pour le déplacement des agents nous permet de vérifier en continu, pendant le déplacement, si un prédateur se trouve à proximité.

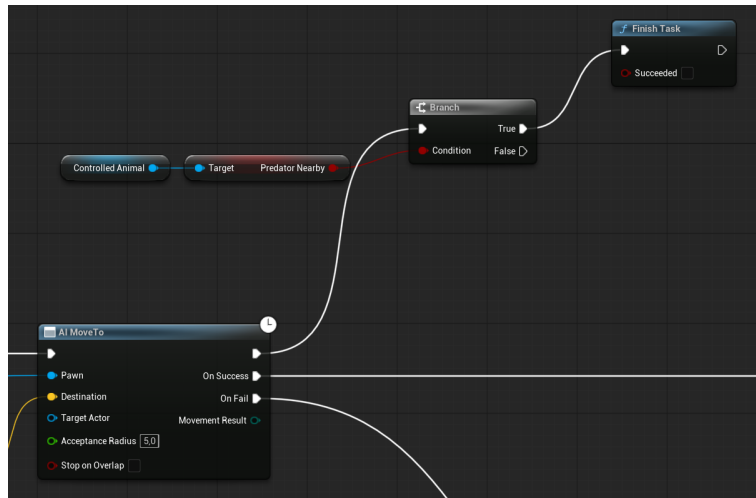


FIGURE 6 – Porte de sortie de la tâche error

Tous ces états forment notre State Tree final (voir figure 7).

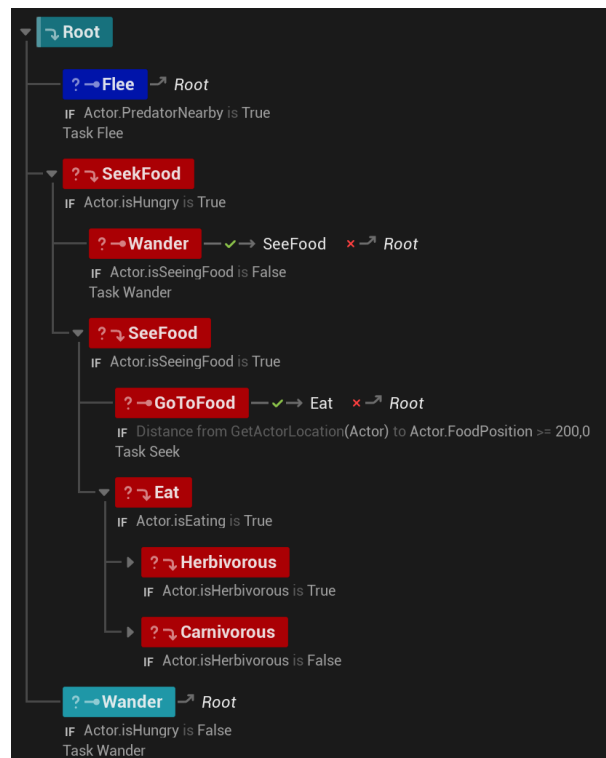


FIGURE 7 – State Tree final

2.3 Axes d'amélioration

Concernant les axes d'amélioration, nous avons vu dans une vidéo traitant de l'utilisation des State Trees que le Gameplay Ability System (GAS) d'Unreal présentait une bonne synergie avec ceux-ci.

2.3.1 Gameplay Ability System

Le GAS pourrait nous permettre de réduire l'utilisation du tick pour les animaux. Actuellement, le tick sert principalement à faire augmenter la jauge de faim, ce qui pourrait être remplacé par des attributs du GAS. Il pourrait également gérer des changements de paramètres, comme la vitesse entre l'état errer et l'état de déplacement vers la nourriture. De plus, le State Tree pourrait s'appuyer sur les Gameplay Tags du GAS plutôt que sur des booléens directement stockés dans l'animal. Enfin, le GAS permettrait d'exécuter des capacités plutôt que des tâches, par exemple pour l'attaque des carnivores, et d'améliorer le système de course-poursuite avec la gestion des points de vie et des dégâts.

3 Simulation d'environnement en temps réel

3.1 Description

Lors de la phase de conception initiale, une partie importante du projet était dédiée à la simulation en temps réel du terrain. Il avait pour objectif de permettre au joueur de modifier dynamiquement la forme des oasis et de leur environnement, sans interrompre la simulation. Les transformations du terrain influencent directement la faune, la végétation et les autres systèmes du jeu.

3.2 Modification de terrain

Objectifs :

- Déformation dynamique des oasis via splines
- Placement de formes prédéfinies
- Mise à jour immédiate des systèmes dépendants :
 - Végétation
 - Eau
 - Faune
- Simulation continue sans génération hors-ligne

Solutions envisagées :

- Utilisation de splines modifiables à l'exécution pour définir la forme des oasis
- Mise à jour de meshes procéduraux à partir de ces splines
- Approche par zones d'influence afin de limiter les recalculs
- Tests de heightfields dynamiques pour la déformation locale du terrain

Résultats / Observations :

- Validation de la faisabilité de :
 - Splines dynamiques
 - Génération de formes d'eau en temps réel (plan avec matériau d'eau)
- Meilleure compréhension :
 - Des coûts de recalcul
 - Des dépendances entre systèmes
- Premiers prototypes fonctionnels à petite échelle
- Identification des limites en termes de performance et de stabilité

Ce qui n'a pas fonctionné / Axes d'amélioration :

- Coût CPU élevé si mises à jour fréquentes
- Difficulté à maintenir :
 - Un rendu visuel stable
 - Des collisions cohérentes
- Problèmes d'interaction avec :
 - Le PCG
 - Le système de navigation

3.3 Méthodologie

3.3.1 Fonctionnement

Le Procedural Mesh Component permet de générer un maillage défini par un ensemble de données géométriques qui comprend les sommets, les triangles, les normales et les coordonnées UV. Il offre un contrôle sur la topologie du terrain, mais implique une gestion manuelle complète de la géométrie et de ses mises à jour, contrairement aux solutions natives de unreal engine.

3.3.2 Représentation et génération du terrain

Le terrain est représenté sous la forme d'une grille de sommets basée sur les axes X/Y/Z. Au moment de la génération, les sommets sont créés selon la résolution définie, les triangles sont construits de façon systématique afin de former la surface, les coordonnées UV sont calculées pour que les textures soient cohérentes et les normales sont générées pour garantir un éclairage correct. Le mesh est ensuite assigné au Procedural Mesh Component afin d'être affiché dans la scène.

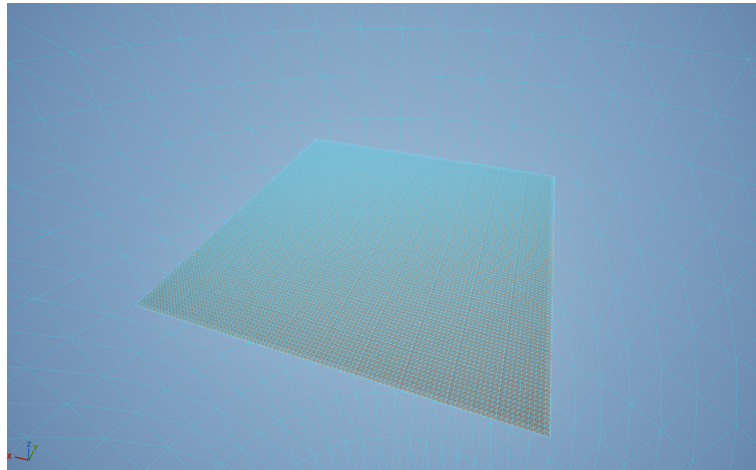


FIGURE 8 – Maillage généré avec une résolution 256 par 256

3.3.3 Déformation dynamique du terrain

La déformation du terrain dépend de la modification des sommets du maillage à partir d'un HeightField généré dynamiquement. Il correspond à une carte de hauteur associée à la grille de sommets du terrain qui permet de stocker et manipuler l'altitude

On déclenche une transformation à partir d'une influence définie, soit à partir d'un rayon ou d'une spline. Cette zone est d'abord appliquée au heightfield, où les valeurs de hauteur sont modifiées en fonction de la distance à la spline ou au masque peint. Il permet d'appliquer des déformations progressives et contrôlées. Exemple : l'ajout, le creusement et des pentes

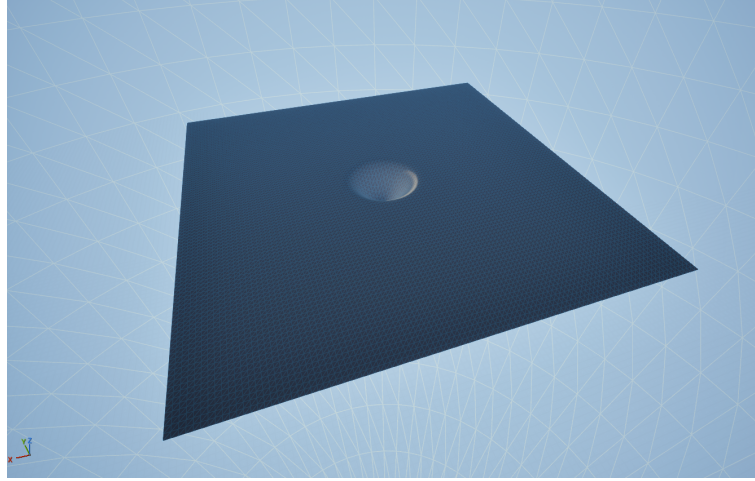


FIGURE 9 – Déformation du terrain par un masque

3.3.4 Interaction avec les oasis et les splines

Les oasis sont définies par des splines modifiables à l'exécution. Elles sont utilisées pour contrôler la forme des zones d'eau et influencent directement la géométrie du terrain.

Lorsque la spline est ouverte, elle est considérée comme une ligne et la modification du terrain est calculée en fonction de la distance entre chaque sommet. Elle permet des déformations progressives le long de la spling

Dans le cas d'une spline fermée, la surface fermée correspondant à l'intérieur de l'oasis. Pour déterminer quels sommets du terrain se trouvent à l'intérieur de cette zone, j'ai utilisé une méthode de type point-in-polygon. Elle consiste à tester si la projection d'un sommet sur le plan du terrain se situe à l'intérieur ou à l'extérieur du contour défini par la spline. Les sommets à l'intérieur de la spline sont alors modifiés.

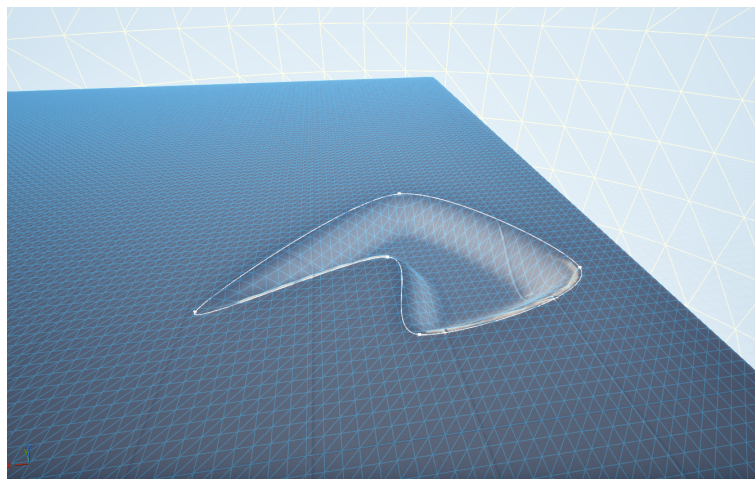


FIGURE 10 – Déformation terrain par une spline fermer

3.3.5 Contraintes de performance

Pour chaque modification de la géométrie le terrain nécessite une mise à jour du maillage qui incluent le recalcul des normales et la reconstruction des collisions. Cette étape était particulièrement coûteuse en termes de performances, notamment lorsque les modifications étaient fréquentes ou que la résolution était très élevée. La synchronisation de ces modifications avec les autres systèmes, comme la génération procédurale et la navigation, est devenue trop complexe.

3.3.6 Analyse critique et limites identifiées

La recherche a révélé plusieurs limites. Les modifications de la géométrie dans une simulation en temps réel à grande échelle, notamment pour un environnement étendu comme un désert, entraînent un coût CPU élevé et est très compliqué mathématiquement à maintenir.

L'absence de mécanismes, comme le Landscape d'Unreal Engine, rend cette approche difficilement extensible. Il faut mettre en place un système de chunk pour éviter le recalcul complet du maillage.

L'intégration du maillage dynamique avec le PCG a également posé problème. Il aurait fallu exposer les données du mesh au PCG pour qu'il soit capables de prendre en charge une géométrie dynamique.

Enfin, la reconstruction du navigation mesh était une autre contrainte. Le système de navigation étant principalement conçu pour des objets statiques, son adaptation à un maillage dynamique nécessitait des ajustements pour qu'il fonctionne. De plus, les calculs des collisions posent des problèmes de stabilité, les animaux chutent sous le terrain au moment des mises à jour.

3.3.7 Décision

En conclusion, les différentes pistes explorées autour de la simulation en temps réel ont été conservées comme une phase d'expérimentation et de recherche, mais écartées pour la version finale du projet.

3.3.8 Runtime Virtual Texture

En plus de l'approche de modification géométrique directe, une autre piste a été étudiée. Cette méthode se basait sur l'utilisation du RVT au landscape, qui enlevait le problème avec le PCG et le système de navigation. La méthode ne change pas véritablement la topographie du paysage, mais seulement sur l'aspect visuel pour donner l'apparence d'un lacs. Cependant, cette approche nécessitait de représenter les contours des oasis définis par les splines dans une texture, puis de créer une heightmap procédurale basée sur ces données pour produire un rendu visuel cohérent, particulièrement en ce qui concerne la gestion des transitions et du falloff des lacs.

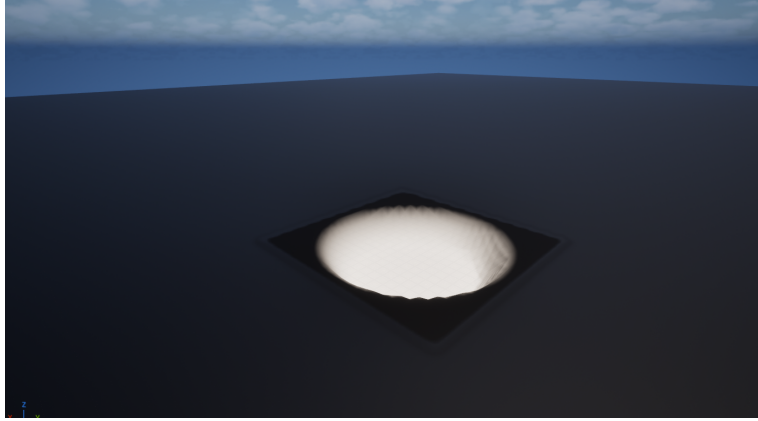


FIGURE 11 – Projection d'une forme sur la texture